

C PROGRAMMING DEBUGGING TOOLS

Project Report

Submitted by: Yug Nirwal

SAP ID: 590033354

Batch: 15

Subject: C programming

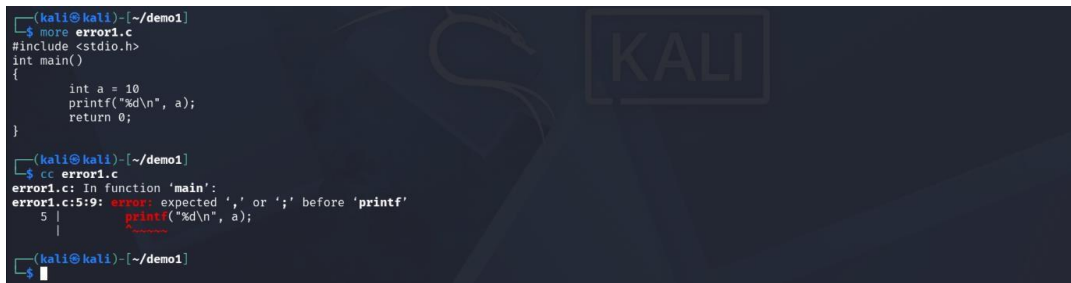
1. Introduction to Errors and Debugging

While writing a C program, mistakes are common. Some errors stop the program from compiling, while others allow it to run but give the wrong answer. Debugging means finding the reason for the problem, understanding what went wrong, and correcting it in a controlled way.

Main Types of Errors

- Syntax error: the code breaks C language rules, such as a missing semicolon or bracket.
- Compilation error: the compiler cannot translate the source code because of issues such as undeclared variables or incorrect data types.
- Linker error: required program parts or function definitions cannot be connected.
- Runtime error: the program fails while running, for example because of division by zero or invalid memory access.
- Logical error: the program runs successfully but produces an incorrect result because the logic or formula is wrong.

A program can compile successfully and still be incorrect because the compiler checks whether the code is valid C; it cannot know what result the programmer intended.

A terminal window with a dark background and a 'KALI' logo watermark. The prompt is '(kali@kali)-[~/demo1]'. The user enters '\$ more error1.c', showing a C program: '#include <stdio.h>', 'int main()', '{', ' int a = 10;', ' printf("%d\n", a);', ' return 0;', '}'. The user then enters '\$ cc error1.c'. The compiler output is: 'error1.c: In function 'main':', 'error1.c:5:9: error: expected ',' or ';' before 'printf'', '5 | printf("%d\n", a);', '|'. The prompt returns to '(kali@kali)-[~/demo1]'.

```
(kali@kali)-[~/demo1]
└─$ more error1.c
#include <stdio.h>
int main()
{
    int a = 10;
    printf("%d\n", a);
    return 0;
}

(kali@kali)-[~/demo1]
└─$ cc error1.c
error1.c: In function 'main':
error1.c:5:9: error: expected ',' or ';' before 'printf'
5 |     printf("%d\n", a);
  |
(kali@kali)-[~/demo1]
└─$
```

Compiler diagnostic for a syntax/compilation error.

2. Compiler Warnings and printf-Based Debugging

Compiler warnings are useful because they point out suspicious code even when compilation succeeds. Using `-Wall` and `-Wextra` helps catch issues such as unused variables and other questionable statements early.

```
(kali㉿kali)-[~/demo1]
$ more error2.c
#include <stdio.h>
int main()
{
    int a;
    printf("Hello\n");
    return 0;
}

(kali㉿kali)-[~/demo1]
$ cc -Wall error2.c
error2.c: In function 'main':
error2.c:4:13: warning: unused variable 'a' [-Wunused-variable]
     4 |     int a;
       |         ^
(kali㉿kali)-[~/demo1]
$
```

Example of a compiler warning.

Using printf() for Debugging

For small programs, `printf()` is one of the easiest debugging methods. Temporary print statements can show variable values, loop counters, function arguments, and intermediate results. This makes it easier to see where the output starts becoming incorrect.

The drawback is that too many debug prints can make the output confusing. For larger programs, a debugger such as GDB is more practical.

```
(kali㉿kali)-[~/demo1]
$ more error3.c
#include <stdio.h>
int main()
{
    int i, sum = 0;
    for(i = 0; i < 5; i++)
    {
        printf("DEBUG: i = %d\n" , i);
        sum = sum + i;
        printf("DEBUG: sum = %d\n" , sum);
    }
    printf("Final Sum = %d\n" , sum);
    return 0;
}

(kali㉿kali)-[~/demo1]
$ cc error3.c

(kali㉿kali)-[~/demo1]
$ ./a.out
DEBUG: i = 0
DEBUG: sum = 0
DEBUG: i = 1
DEBUG: sum = 1
DEBUG: i = 2
DEBUG: sum = 3
DEBUG: i = 3
DEBUG: sum = 6
DEBUG: i = 4
DEBUG: sum = 10
Final Sum = 10
```

Tracking values with `printf()` statements.

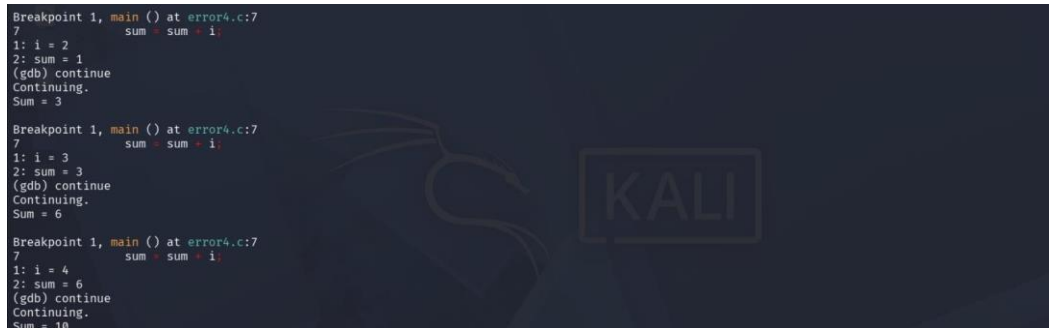
3. Introduction to GDB

GDB stands for GNU Debugger. It lets us pause a running program, inspect variables, move through the code step by step, check the call stack, and investigate crashes. It is especially helpful when printf() statements are no longer enough.

For source-level debugging, the program is normally compiled with the -g option. Warning flags can also be used at the same time: cc -g -Wall -Wextra program.c -o program.

Breakpoints and Useful Commands

- break (b): places a breakpoint at a function or source line.
- run (r): starts or restarts the program.
- print (p): displays the current value of a variable or expression.
- next (n): executes the next line without entering a called function.
- step (s): enters a called function so its internal statements can be checked.
- continue (c): resumes execution until the next stopping point.
- info locals: displays local variables in the current stack frame.
- backtrace (bt): shows the sequence of function calls that led to the current point.



```
Breakpoint 1, main () at error4.c:7
7:      sum = sum + 1;
1: i = 2
2: sum = 1
(gdb) continue
Continuing.
Sum = 3

Breakpoint 1, main () at error4.c:7
7:      sum = sum + 1;
1: i = 3
2: sum = 3
(gdb) continue
Continuing.
Sum = 6

Breakpoint 1, main () at error4.c:7
7:      sum = sum + 1;
1: i = 4
2: sum = 6
(gdb) continue
Continuing.
Sum = 10
```

GDB session showing a logical-error investigation.

4. Debugging Logical Errors with GDB

Logical errors can be difficult because the program may run normally. A good method is to stop near the suspicious statement, inspect the current values, and then move through the code one line at a time. For example, if a loop should calculate $1+2+3+4+5$ but gives 10, checking the loop counter can reveal that the final iteration is being skipped.

```
Reading symbols from ./error5...
(gdb) list
4      int result1;
5      result1 = x * x;
6      return result1;
7  }
8  int main()
9  {int result, n = 5;
10     result = square n;
11     printf "%d\n", result;
12     return 0;
13 }
(gdb) break main
Breakpoint 1 at 0x1156: file error5.c, line 9.
(gdb) run
Starting program: /home/kali/demo1/error5
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-l
inux-gnu/libthread_db.so.1".

Breakpoint 1, main () at error5.c:9
9      (int result, n = 5;
(gdb) next
10     result = square n;
(gdb) next
11     printf "%d\n", result;
(gdb) print result
$1 = 25
(gdb)
```

Inspecting program execution in GDB.

next vs. step

Both commands move execution forward. next runs a function call without entering its body, while step enters the function and lets us inspect its parameters, local variables, and statements. If the function itself may contain the bug, step is the better choice.

```
Breakpoint 1, main () at error5.c:9
9      (int result, n = 5;
(gdb) next
10     result = square n;
(gdb) step
square (x=5) at error5.c:5
5      result1 = x * x;
(gdb) print x
$1 = 5
(gdb) next
6      return result1;
(gdb) print result
X No symbol "result" in current context.
(gdb) print result1
$2 = 25
(gdb)
```

Function debugging using next and step.

5. Runtime Errors and Backtrace

Runtime errors happen after a program starts executing. A common example is division by zero. When a program crashes, GDB can stop at the failure and backtrace (bt) can show the chain of function calls that led to it.

```
(kali@kali)-[~/demo1]
$ cc -g error6.c -o error6

(kali@kali)-[~/demo1]
$ gdb ./error6
GNU gdb (Debian 17.2-1+b1) 17.2
Copyright (C) 2025 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://
/gnu.org/licenses/gpl.html>
This is free software: you are free to change and
redistribute it.
There is NO WARRANTY, to the extent permitted by l
aw.
Type "show copying" and "show warranty" for detail
s.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration detail
s.
For bug reporting instructions, please see:
--Type <RET> for more, q to quit, c to continue wi
thout paging--
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resour
ces online at:
<http://www.gnu.org/software/gdb/documentation
/>.
For help, type "help".
Type "apropos word" to search for commands related
to "word" ...
Reading symbols from ./error6...
```

Runtime error/crash being examined in GDB.

Understanding Stack Frames

In a backtrace, frame #0 is normally the function where the crash occurred. The following frames show which functions called it. This helps trace the problem back from the crash location to the higher-level code that caused it.

```
(gdb) list
6      int calculate(int x)
7      {
8          return divide(x,0);
9      }
10     int main()
11     {
12         int result = calculate(10);
13         printf("Result:%d\n", result);
14         return 0;
15     }
(gdb) run
Starting program: /home/kali/demo1/error6
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-l
inux-gnu/libthread_db.so.1".

Program received signal SIGFPE, Arithmetic excepti
on.
0x000055555555147 in divide (a=10, b=0)
    at error6.c:4
4      return a/b;
(gdb) backtrace
#0  0x000055555555147 in divide (a=10, b=0)
    at error6.c:4
#1  0x000055555555166 in calculate (x=10)
    at error6.c:8
#2  0x00005555555517a in main () at error6.c:12
(gdb) frame 0
#0  0x000055555555147 in divide (a=10, b=0)
    at error6.c:4
```

Backtrace and stack-frame information.

6. Array Out-of-Bounds Errors

C does not automatically check normal array indexes. If an array is declared as `int a[5]`, its valid indexes are 0 to 4. Accessing `a[5]` goes outside the array and causes undefined behaviour. It may overwrite unrelated memory or lead to a crash.

To debug this problem, check the loop condition, inspect the index value, and stop execution near the array access. The correct loop should keep `i` below the array size.

```
(gdb) frame 0
#0  0x000055555555147 in divide (a=10, b=0)
    at error6.c:4
4      return a/b;
(gdb) print b
$1 = 0
(gdb) frame 1
#1  0x000055555555166 in calculate (x=10)
    at error6.c:8
8      return divide(x,0);
(gdb) print x
$2 = 10
(gdb) █
```

Array index/boundary debugging example.

7. Segmentation Faults (SIGSEGV)

A segmentation fault usually means the program tried to access memory that it was not allowed to use. One common cause is dereferencing a NULL pointer. GDB can show the signal, stop at the failing line, and allow the pointer value to be inspected.

```
to "word" ...
Reading symbols from ./error7...
(gdb) list
1      #include <stdio.h>
2      int main()
3      {
4          int a[5];
5          int i;
6          for(i = 0; i ≤ 5; i++)
7          {
8              scanf("%d", &a[i]);
9          }
10         return 0;
(gdb) break 8
Breakpoint 1 at 0x114a: file error7.c, line 8.
(gdb) run
Starting program: /home/kali/demo1/error7
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-l
inux-gnu/libthread_db.so.1".

Breakpoint 1, main () at error7.c:8
8          scanf("%d", &a[i]);
(gdb) print &a[0]
$1 = (int *) 0x7ffffffdd10
(gdb) print &a[4]
$2 = (int *) 0x7ffffffdd20
(gdb) print &a[5]
$3 = (int *) 0x7ffffffdd24
(gdb) |
```

GDB showing a segmentation fault.

The safer approach is to make sure a pointer refers to valid storage before dereferencing it. For example, a pointer can be assigned the address of a valid variable before using *p.

```
Find the GDB manual and other documentation resour
ces online at:
<http://www.gnu.org/software/gdb/documentation
/>.
For help, type "help".
Type "apropos word" to search for commands relat
ed to "word" ...
Reading symbols from ./error8...
(gdb) list
1      #include <stdio.h>
2      int main()
3      {
4          int *p = NULL;
5          *p = 10;
6          printf("%d\n", *p);
7          return 0;
8      }
(gdb) run
Starting program: /home/kali/demo1/error8
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-l
inux-gnu/libthread_db.so.1".

Program received signal SIGSEGV, Segmentation faul
t.
0x000055555555514d in main () at error8.c:5
5          *p = 10;
(gdb) print p
$1 = (int *) 0x0
(gdb) print *p
✗ Cannot access memory at address 0x0
```

Corrected pointer approach / debugging output.

8. Simple Debugging Strategy

A systematic approach saves time and avoids random changes to the program.

- Compare the expected output with the actual output.
- Reproduce the problem using the same input or conditions.
- Compile with useful warnings and debugging information (-Wall -Wextra -g).
- Focus on the smallest suspicious function, loop, condition, or statement.
- Inspect values using printf() or GDB commands such as break, print and info locals.
- Make one small correction at a time.
- Recompile and test again using the same case.
- Finally, test boundary cases such as zero, negative values, and first/last loop iterations.

Conclusion

Different errors appear at different stages, so there is no single debugging method for every problem. Compiler messages and warnings are useful for code-level mistakes, printf() works well for simple tracing, and GDB is better for deeper logical and runtime problems. The key idea is to debug using evidence: reproduce the issue, inspect the program state, make a small fix, and test again.